



# Sequential Decision Making under Uncertainty

CS4246/5446

Reinforcement Learning and Sequential Decision Making

Version 1.0



# What we have learned so far...

- Symbolic (Classical) Planning
  - Sequential, deterministic, fully observable
- Utility Theory
  - Episodic or sequential, stochastic, fully or partially observable
- Up Next: Decision Making Under Uncertainty and Over Time
  - Sequential, stochastic, fully or partially observable
  - Markov decision process
  - Reinforcement learning



# Topics

- Markov Decision Process (17.1)
  - Model formulation and solution
  - Bellman Equation and Q-function
  - Solution algorithm: Value iteration (17.2.1)
  - Solution algorithm: Policy iteration (17.2.2)
- Reinforcement Learning: Reward Based Learning and Planning
- Prediction (Learning) - Passive RL
  - Adaptive dynamic programming
  - Monte Carlo learning: Direct utility estimation
  - Temporal difference learning
- Control (Planning) - Active RL
  - Active adaptive dynamic programming
  - Exploration vs exploitation
  - Monte Carlo Control
  - Temporal difference Control: Q-learning and SARSA



# Solving Sequential Decision Problem

- **Decision Problem or Model**
  - Appropriate abstraction of states, actions, uncertain effects, and goals (wrt costs and values or preferences) and time horizon + observations (through sensing)
- **Decision Algorithm**
  - Input: a problem
  - Output: an optimal solution over time horizon
- **Decision Solution**
  - An action sequence or solution from an initial state to the goal state(s)
    - An optimal action sequence; OR
    - An optimal policy that specifies “best” action in each state wrt to costs or values or preferences
  - (Optional) A goal state that satisfies certain properties



# Markov Decision Process

Foundational sequential decision modeling and solution framework

# Markov Decision Process (MDP)

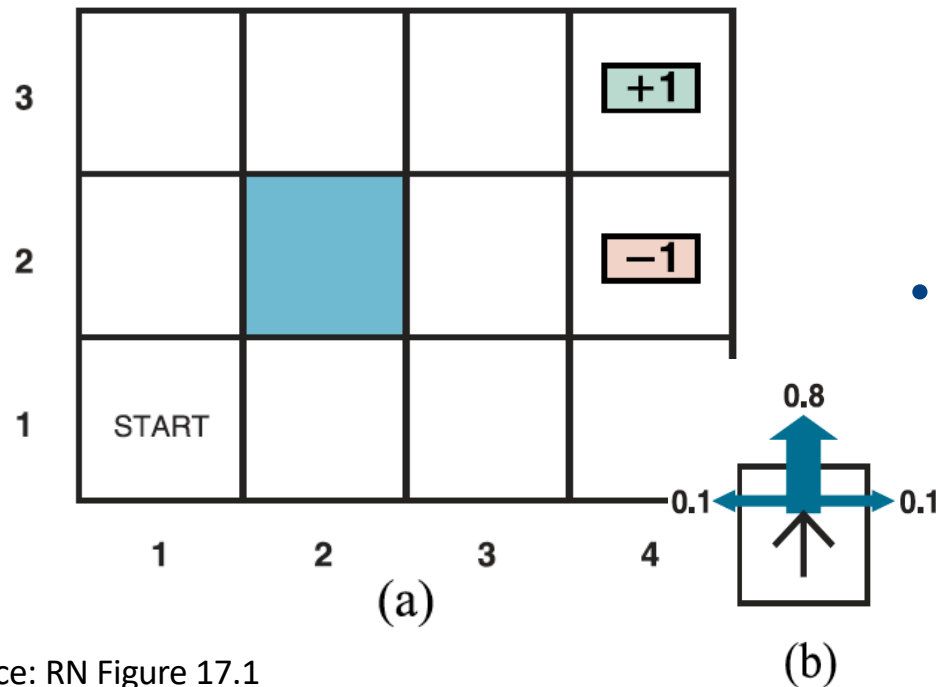
- Formally:

- An MDP  $M \triangleq (S, A, T, R, \gamma)$  consists of:
- A set  $S$  of states
- A set  $A$  of actions
- A transition function  $T: S \times A \times S \rightarrow [0,1]$  that satisfies the **Markov property** such that:

$$\forall s \in S, \forall a \in A: \sum_{s' \in S} T(s, a, s') = \sum_{s' \in S} P(s'|s, a) = 1$$

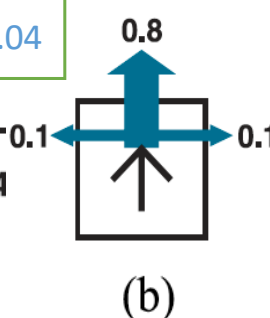
- A reward function  $R: S \rightarrow \mathfrak{R}$  or  $R: S \times A \rightarrow \mathfrak{R}$  or  $R: S \times A \times S \rightarrow \mathfrak{R}$
- A discount factor  $0 \leq \gamma \leq 1$
- Solution is a policy – a function to recommend an action in each state:  $\pi: S \rightarrow A$ 
  - Solution involves careful balancing of risk and reward

# Example: Navigation in Grid World



Source: RN Figure 17.1

- Fully observable 4 x 3 environment
  - Begin in START state  $s_0$
  - One action  $a$  per time step.
  - Terminate when reaching goal states  $s_g$
- Uncertain action effects:
  - Example: Up, Down, Left, Right:
  - 0.8 - correct direction
  - 0.1 - perpendicular to the left
  - 0.1 - perpendicular to the right



## Sequential Decision Making

- ### 0.1 Utility function: (State utility)



# Solving MDPs

- What does a solution look like?
  - A policy  $\pi: S \rightarrow A$  maps each state  $s$  to an action  $a$ .
  - The *quality* of a policy is measured by its expected utility over the state sequences it generates.
  - The **optimal policy**  $\pi^*$  maximizes expected utility. (MEU Principle)
- An MDP agent uses  $\pi^*$ :
  - It observes the current state  $s$
  - Executes the action  $a^* = \pi^*(s)$
  - The policy defines how the agent behaves.



# Notes: State Utility and Optimal Policy

- Utility of a state  $s$  (Value of a state  $s$ ):
  - Best achievable expected utility starting from  $s$ :

$$U(s) = U^{\pi^*}(s) = \max_{\pi} U^{\pi}(s)$$

- Bellman Optimality Equation:

$$U(s) = \max_{a \in A(s)} \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma U(s')]$$

- Optimal policy (Maximum Expected Utility Principle):

$$\pi^*(s) = \operatorname{argmax}_{a \in A(s)} \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma U(s')]$$

- Interpretation:
  - Utility of a state is the maximum expected immediate reward plus discounted utility of successor states.

# Solving Bellman Optimality Equation

Numbers are  $U(s)$

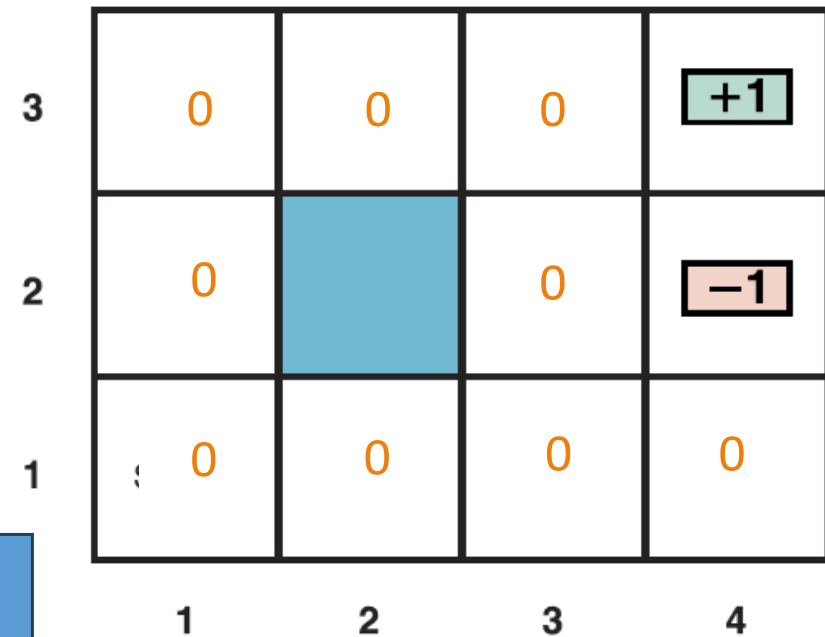
Initialize:  $U(s) = 0$

How to compute  
the  $U(s)$  numbers?

$$U_{i+1}(s) \leftarrow \max_{a \in A(s)} \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma U_i(s')]$$

Bellman update

Utility of a state  
is the maximum expected immediate reward  
plus discounted utility of successor states



(a)

# Notes: Value Iteration

- Iterative, repeated Bellman updates of state utilities:

1. Initialize utilities arbitrarily (e.g.,  $U(s) = 0$ )

2. Update utilities of all states using:

$$U_{i+1}(s) \leftarrow \max_{a \in A(s)} \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma U_i(s')]$$

Bellman update



3. Repeat until values converge (changes < threshold  $\epsilon$ )

4. Extract optimal policy from final utilities:

$$\pi^*(s) = \operatorname{argmax}_{a \in A(s)} \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma U(s')]$$

- Key points:

- Convergence: Approximates utilities until convergence to the unique Bellman solution.
- Optimality: Extracted policy is guaranteed to be optimal.
- Complexity: Each iteration is  $O(|S|^2 |A|)$ ; total iterations depends on  $\epsilon$  and  $\gamma$ .

# Example: Solution by Value Iteration

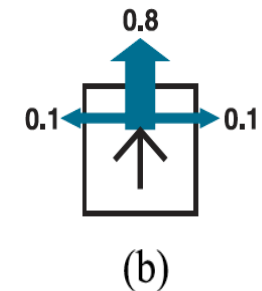
$$R(s) = -0.04$$

Numbers are  $U(s)$

How to compute the numbers?

What is the optimal policy?

|   |        |        |        |        |
|---|--------|--------|--------|--------|
| 3 | 0.8516 | 0.9078 | 0.9578 | +1     |
| 2 | 0.8016 |        | 0.7003 | -1     |
| 1 | 0.7453 | 0.6953 | 0.6514 | 0.4279 |
|   | 1      | 2      | 3      | 4      |



For  $U(1,1)$ :

$$U_{i+1}(s) \leftarrow \max_{a \in A(s)} \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma U_i(s')]$$

$$\begin{aligned} \max \{ & [0.8(-0.04 + \gamma U(1, 2)) + 0.1(-0.04 + \gamma U(2, 1)) + 0.1(-0.04 + \gamma U(1, 1))], & (U) \\ & [0.9(-0.04 + \gamma U(1, 1)) + 0.1(-0.04 + \gamma U(1, 2))], & (L) \\ & [0.9(-0.04 + \gamma U(1, 1)) + 0.1(-0.04 + \gamma U(2, 1))], & (D) \\ & [0.8(-0.04 + \gamma U(2, 1)) + 0.1(-0.04 + \gamma U(1, 2)) + 0.1(-0.04 + \gamma U(1, 1))] \} & (R) \end{aligned}$$

$$\pi^*((1,1)) = \text{Up}$$

# Notes: Policy Iteration

- Iterative Steps:

1. Initialize an arbitrary policy  $\pi_0$ .

2. Policy Evaluation: Solve for utilities  $U^\pi(s)$  under  $\pi_i$ :

$$U^{\pi_i}(s) = U_i(s) = \sum_{s'} P(s'|s, \pi_i(s)) [R(s, \pi_i(s), s') + \gamma U_i(s')]$$

Bellman expectation equation

- For large state spaces, use iterative approximation:

$$U_{i+1}(s) \leftarrow \sum_{s'} P(s'|s, \pi_i(s)) [R(s, \pi_i(s), s') + \gamma U_i(s')]$$

3. Policy Improvement: Update policy  $\pi_{i+1}$  based on current utilities:

$$\pi_{i+1}(s) = \arg \max_a \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma U_i(s')]$$

4. Repeat until the policy stabilizes (no change between iterations).

- Key Points:

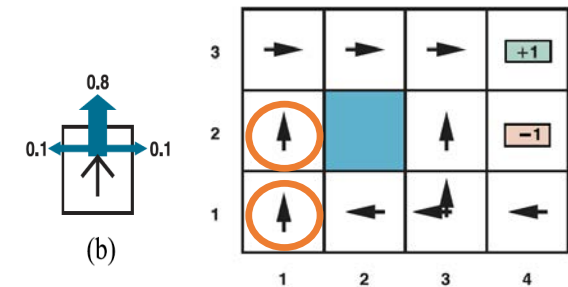
- Convergence: Terminates in a finite number of iterations.
- Optimality: Final policy is guaranteed to be optimal.
- Complexity: Policy evaluation solves  $|S|$  linear equations ( $O(n^3)$  for  $|S| = n$ ); iterative evaluation used for large state spaces

# Example: Solution by Policy Iteration

- Step 1: Policy evaluation:  
(Bellman expectation equation)

$$U_i(s) = \sum_{s'} P(s'|s, \pi_i(s)) [R(s, \pi_i(s), s') + \gamma U_i(s')] \\ = R(s) + \gamma \sum_{s'} P(s'|s, \pi_i(s)) U_i(s')$$

NO max operator



Assume:  $\pi_i(s)$

- $U_i(1,1) = 0.8[-0.04 + U_i(1,2)] + 0.1[-0.04 + U_i(2,1)] + 0.1[-0.04 + U_i(1,1)]$
- $U_i(1,2) = 0.8[-0.04 + U_i(1,3)] + 0.2[-0.04 + U_i(1,2)]$
- $\vdots$

Source: RN 17.2.2

# Example: Solution by Policy Iteration

## • Step 2: Policy improvement:

- For all the states, to find best policy:

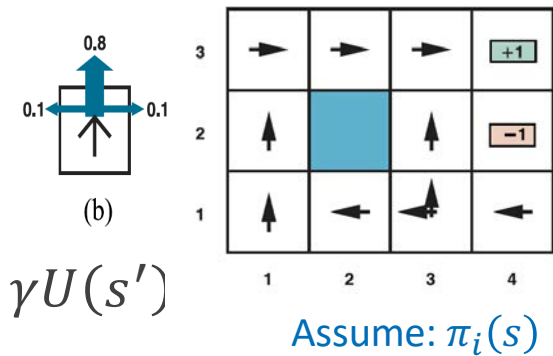
$$\pi^*(s) = \operatorname{argmax}_{a \in A(s)} \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma U(s')]$$

- In state (1, 1):

- Check if any action  $a$  when applied to  $s$  is better than current policy  $\pi_i(s)$ ;
- Update if necessary

$$\pi_{i+1}(1,1) = \operatorname{argmax}_{a \in A(s)} \sum_{s'} P(s'| (1,1), a) [R((1,1), a, s') + \gamma U(s')]$$

$$\begin{aligned} & \{ [0.8(-0.04 + \gamma U(1,2)) + 0.1(-0.04 + \gamma U(2,1)) + 0.1(-0.04 + \gamma U(1,1))], & (U) \\ & [0.9(-0.04 + \gamma U(1,1)) + 0.1(-0.04 + \gamma U(1,2))], & (L) \\ & [0.9(-0.04 + \gamma U(1,1)) + 0.1(-0.04 + \gamma U(2,1))], & (D) \\ & [0.8(-0.04 + \gamma U(2,1)) + 0.1(-0.04 + \gamma U(1,2)) + 0.1(-0.04 + \gamma U(1,1))] \} & (R) \end{aligned}$$

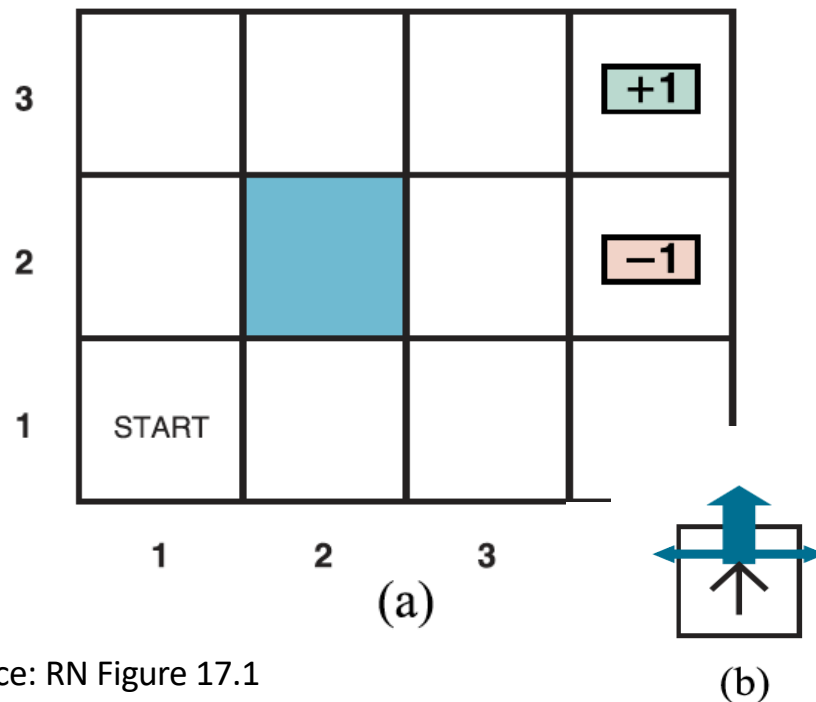




# Reinforcement Learning

Combining planning + learning in sequential decision making

# Example: Navigation in Grid World

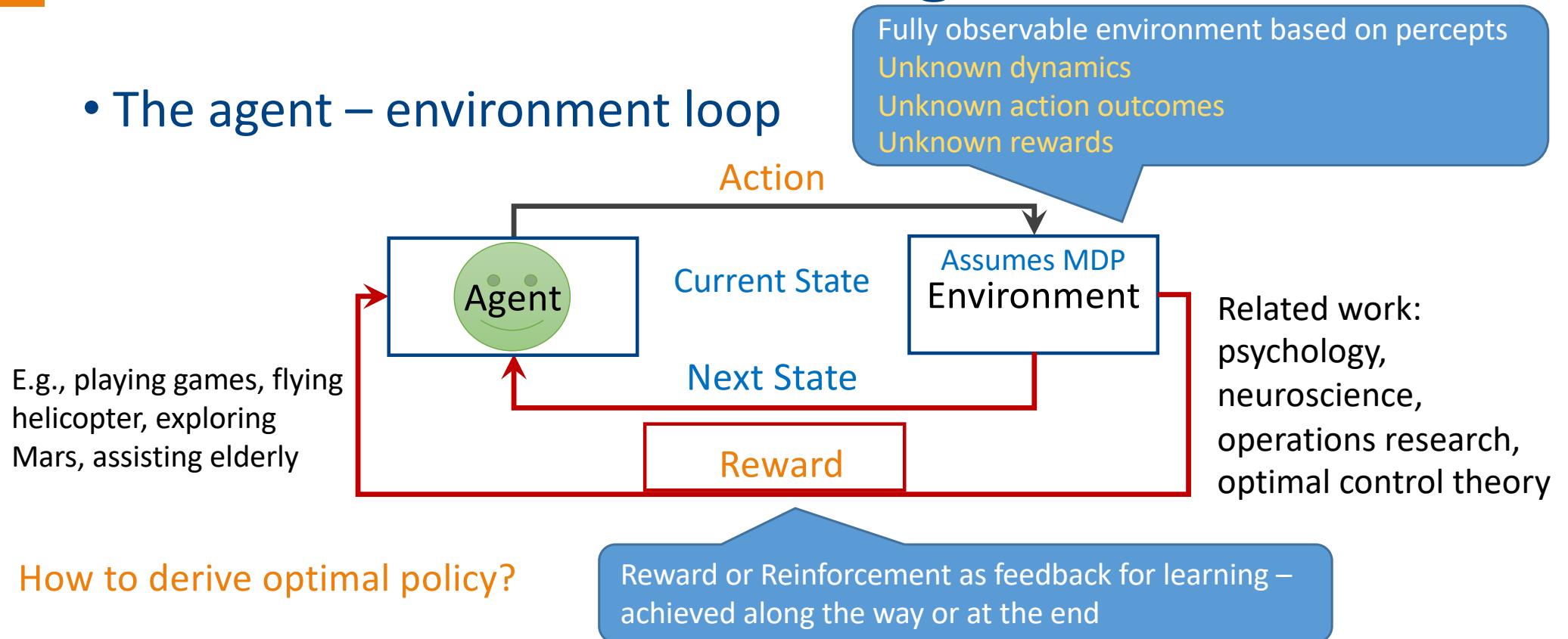


Source: RN Figure 17.1

- Transition model:
  - Unknown
- Reward model:
  - Unknown
- How to find optimal policy  $\pi^*$ ?

# Reinforcement Learning

- The agent – environment loop





# Reinforcement Learning Videos

- Stanford Autonomous Helicopter
  - [https://youtu.be/M-QUkgk3HyE?si=7yx8E0UeObDJk\\_IJ](https://youtu.be/M-QUkgk3HyE?si=7yx8E0UeObDJk_IJ)
- Run Away from Zombies!
  - <https://youtu.be/TWBSSCYnEOM?si=iQdbr6EXEMnJdZYA>
- RL in Starcraft II
  - <https://youtu.be/gEyBzcPU5-w>
- Autonomous robot control
  - <https://youtu.be/KyA2uTIQfxw>

# Notes: Reinforcement Learning

E.g., playing chess, flying helicopter,  
exploring Mars, assisting elderly

- Decision making in complex and uncertain environments
  - Learning to behave in unfamiliar environment, given percepts and *occasional* rewards
- Assume:
  - Environment is a Markov Decision Process (MDP)
  - Optimal policy – policy that maximizes the expected total reward
- Reinforcement learning
  - Unknown environment model reward function
  - Use observed rewards to learn an optimal policy for the environment
  - Related work: psychology, neuroscience, operations research, optimal control theory.

Relies on the Markov property

# Types of RL Agents

Assume:  
Environment is an MDP  
Optimal policy maximizing total rewards

- Reinforcement learning
  - [MB] Model-based: Learns transition model (and reward model) to solve problem
  - [MF] Model-free: Does not (need to) learn transition model to solve problem
- Passive learning agent  $\pi \rightarrow U^\pi$ 
  - Learning - Prediction/evaluation/value estimation problem
  - Fixed policy determining behavior
  - Learns utility functions or action-utility functions (Q-functions)
  - sf policy evaluation in policy iteration
- Active learning agent  $\pi^*$ 
  - Planning - Control problem
  - Learning how to behave – balancing exploration vs exploitation
  - Plans or determines actions to take
  - sf value iteration



# Notes: Q-Function

- Q-Function (Action-Utility Function)
  - State utility  $U(s)$  is defined by the Bellman Optimality Equation.
  - To select actions, it is useful to evaluate state–action pairs directly.
- Definition (Bellman Optimality Equation for Q):

$$Q(s, a) = \sum_{s'} P(s'|s, a)[R(s, a, s') + \gamma U(s')]$$

- State utility:  $U(s) = \max_a Q(s, a)$
  - Optimal policy:  $\pi^*(s) = \operatorname{argmax}_a Q(s, a)$
- Interpretation:
  - $Q(s, a)$  = expected utility of taking action  $a$  in state  $s$ , then following the optimal policy.



## Passive Reinforcement Learning $\pi \rightarrow U^\pi$

Predict utility function (action-utility function) of state given a fixed policy

# Notes: Passive Learning

- Main ideas

- Agent follows a fixed policy  $\pi$  for multiple trials (episodes)
- Starts from an initial state  $s_0$  and continues until reaching a terminal state
- Observes states and rewards while executing  $\pi$
- Learns expected utilities under  $\pi$ :

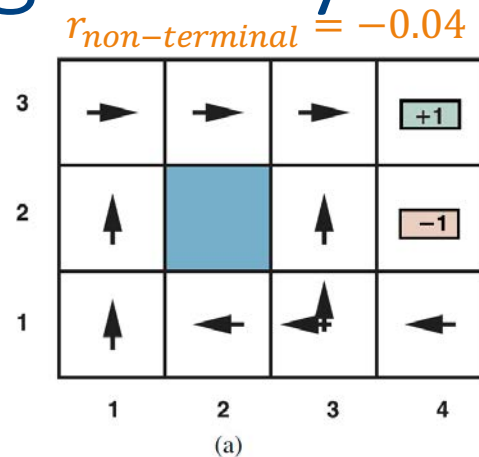
$$U^\pi(s) = \mathbb{E}_\pi \left[ \sum_{t=0}^{\infty} \gamma^t R(S_t, \pi(S_t), S_{t+1}) \mid S_0 = s \right]$$

$$Q^\pi(s, a) = \mathbb{E}_\pi \left[ \sum_{t=0}^{\infty} \gamma^t R(S_t, A_t, S_{t+1}) \mid S_0 = s, A_0 = a \right]$$

Discounted  
Rewards received

- This is policy evaluation in policy iteration.

# Learning Utility Function



Source: RN Figure 22.1

- The policy  $\pi$  is fixed:
    - Agent always executes  $\pi(s)$  in state  $s$
  - Goal: Learn utility function or value function  $U^\pi(s)$  from observations
    - Unknown transition model,  $P(s'|s, a)$
    - Unknown reward function,  $R(s)$  or  $R(s, a, s')$
- } If known, simply do policy evaluation!

# Trials, Returns, and Expected Returns

- Trials

- Also called state-action sequences/runs/trajectories/episodes

- Returns ( $G_t$ )

- Discounted sum of rewards of a trial from time  $t$  onwards

$$G_t = R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \dots$$

- Example:  $G_t = -0.04 + \gamma(-0.04) + \gamma^2(-0.04) + \dots + \gamma^n(1)$

- Expected Returns

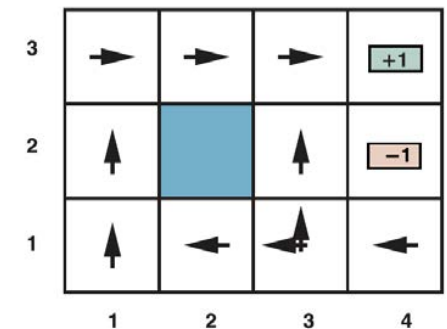
- Utility / value of a state (under policy  $\pi$ ):

$$U^\pi(s) = E_\pi[\sum_{t=0}^{\infty} \gamma^t R(S_t) | S_0 = s] = E_\pi[G_0 | S_0 = s]$$

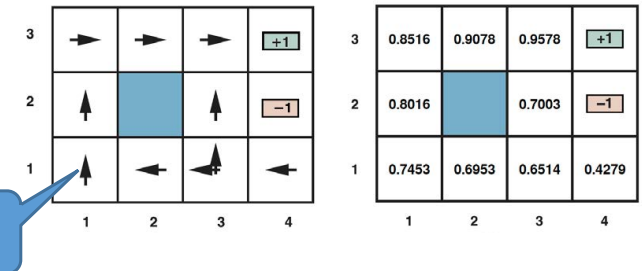
- Note: Q-function (Action-utility function)  $Q(s, a)$

- Expected total discounted reward if action  $a$  is taken in state  $s$ , followed by optimal behavior
- Related to state utilities:  $U(s) = \max_a Q(s, a)$

$$r_{\text{non-terminal}} = -0.04$$



# Learning Utility Function



- Agent executes a series of trials / episodes / runs using  $\pi$ , e.g.,

- Utility or value of a state under  $\pi$ :

- Expected total reward from that state onwards following  $\pi$
- Also called expected reward-to-go / expected return

$$U^\pi(s) = E \left[ \sum_{t=0}^{\infty} \gamma^t R(S_t) \mid S_0 = s \right] = E \left[ \sum_{t=0}^{\infty} \gamma^t R(S_t, \pi(S_t), S_{t+1}) \mid S_0 = s \right]$$

$G_t$

Return  $G_t$ :  
Sum of discounted rewards  
of a trial

From now on:  
Assume  $\gamma = 1$



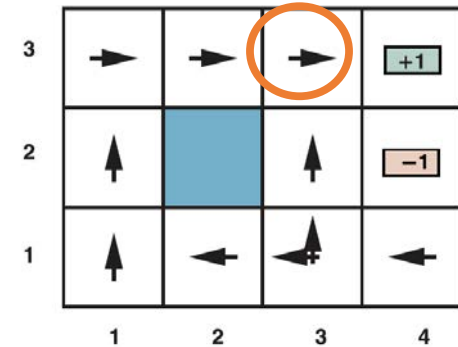
# Learning Utility Function

- Unknown transition dynamics, unknown reward functions
- **Model-based**
  - Learn the transition and reward functions, then solve the MDP ( $U^\pi$ )
  - Examples: Adaptive Dynamic Programming (ADP)
- **Model-free**
  - Directly learn the utility function ( $U^\pi$ )
  - Examples: Monte Carlo (MC) Learning, Temporal Difference (TD) Learning

$$\begin{array}{l} (1, 1)_{-.04} \rightsquigarrow (1, 2)_{-.04} \rightsquigarrow (1, 3)_{-.04} \rightsquigarrow (1, 2)_{-.04} \rightsquigarrow (1, 3)_{-.04} \rightsquigarrow (2, 3)_{-.04} \rightsquigarrow (3, 3)_{-.04} \rightsquigarrow (4, 3)_{+1} \\ (1, 1)_{-.04} \rightsquigarrow (1, 2)_{-.04} \rightsquigarrow (1, 3)_{-.04} \rightsquigarrow (2, 3)_{-.04} \rightsquigarrow (3, 3)_{-.04} \rightsquigarrow (3, 2)_{-.04} \rightsquigarrow (3, 3)_{-.04} \rightsquigarrow (4, 3)_{-1} \\ (1, 1)_{-.04} \rightsquigarrow (2, 1)_{-.04} \rightsquigarrow (3, 1)_{-.04} \rightsquigarrow (3, 2)_{-.04} \rightsquigarrow (4, 2)_{-1} . \end{array}$$

What is the transition probability function from  $s = (3, 3)$ ?

## To compute:



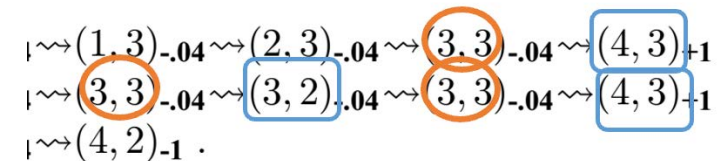
# Adaptive Dynamic Programming (ADP)

- Learn model from experience:
  - Estimate transition probabilities  $P(s'|s, a)$  by counting and normalizing observed outcomes.
  - Learn rewards  $R(s, a, s')$  when entering state  $s'$ .
- Evaluate policy using learned model:

$$U^\pi(s) = \sum_{s'} P(s'|s, \pi(s)) [R(s, \pi(s), s') + \gamma U^\pi(s')] ]$$

- Notes:

- Solve  $|S|$  linear equations in  $O(|S|^3)$  time
- Learning  $P(s'|s, a)$  is supervised learning:  
Input: a state–action pair  $(s, a)$ , target: observed next state  $s'$ .
- Learning  $R(s, a, s')$  is supervised learning:  
Input: a transition  $(s, a, s')$ , target: observed reward.
- ADP enforces global consistency among utilities using Bellman expectation equations.



# Notes: ADP-Learner

**function** PASSIVE-ADP-LEARNER(*percept*) **returns** an action

**inputs:** *percept*, a percept indicating the current state  $s'$  and reward signal  $r$

**persistent:**  $\pi$ , a fixed policy

*mdp*, an MDP with model  $P$ , rewards  $R$ , actions  $A$ , discount  $\gamma$

*U*, a table of utilities for states, initially empty

*N* <sub>$s'|s,a$</sub> , a table of outcome count vectors indexed by state and action, initially zero

*s*, *a*, the previous state and action, initially null

Using tabular representation {

**if**  $s'$  is new **then**  $U[s'] \leftarrow 0$

**if**  $s$  is not null **then**

    increment  $N_{s'|s,a}[s, a][s']$

$R[s, a, s'] \leftarrow r$

    add  $a$  to  $A[s]$

$\mathbf{P}(\cdot \mid s, a) \leftarrow \text{NORMALIZE}(N_{s'|s,a}[s, a])$  ← Maximum likelihood estimation (MLE)

$U \leftarrow \text{POLICYEVALUATION}(\pi, U, mdp)$

$s, a \leftarrow s', \pi[s']$

    return  $a$

Reward function  $R$

Transition function  $T$

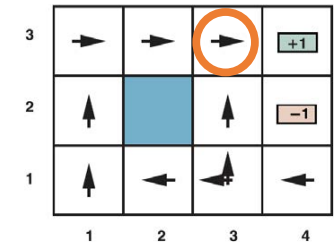
Source: RN Figure 22.2



# Learning Utility Function

- Unknown transition dynamics, unknown reward functions
- Model-based
  - Learn the transition and reward functions, then solve the MDP ( $U^\pi$ )
  - Examples: Adaptive Dynamic Programming (ADP)
- **Model-free**
  - Directly learn the utility function ( $U^\pi$ )
  - Examples: Monte Carlo Learning, Temporal Difference (TD) Learning

# Example: Monte Carlo Learning



$(1,1) \xrightarrow{-0.04} (1,2) \xrightarrow{-0.04} (1,3) \xrightarrow{-0.04} (1,2) \xrightarrow{-0.04} (1,3) \xrightarrow{-0.04} (2,3) \xrightarrow{-0.04} (3,3) \xrightarrow{-0.04} (4,3) +1$   
 $(1,1) \xrightarrow{-0.04} (1,2) \xrightarrow{-0.04} (1,3) \xrightarrow{-0.04} (2,3) \xrightarrow{-0.04} (3,3) \xrightarrow{-0.04} (3,2) \xrightarrow{-0.04} (3,3) \xrightarrow{-0.04} (4,3) +1$   
 $(1,1) \xrightarrow{-0.04} (2,1) \xrightarrow{-0.04} (3,1) \xrightarrow{-0.04} (3,2) \xrightarrow{-0.04} (4,2) -1$

- Each trial provides a “sample” of expected return for each state visited

- In the first trial, sample total reward:

- For  $(1,1) =$
- For  $(1,2) =$

- Overall:

- $U^\pi(1,1) =$
- $U^\pi(1,2) =$

$$U^\pi(s) \leftarrow \frac{1}{N(s)} \sum_{i=1}^{N(s)} G_t^{(i)}; \quad G_t^{(i)} = \sum_{k=0}^{\infty} \gamma^k R(S_{t+k})$$

- $N(s)$  = number of times state  $s$  has been visited
- $G_t^{(i)}$  = return from the  $i$ -th visit to  $s$



# Monte Carlo Learning

- Goal: Estimate state or state–action utilities under a fixed policy.
- Method: Run trials (episodes), then update utilities using the observed return  $G$
- Learning: Update by averaging returns across visits to each state or (state, action).
- Key idea:

The return  $G$  (total discounted reward) is an **unbiased estimate** of true utility:

$$\mathbb{E}[G] = U^\pi(s) \text{ or } Q^\pi(s, a)$$

- Properties:
  - No model required
  - High variance, slow convergence → Does not enforce Bellman consistency during learning
  - Updates only after each episode completes
  - Useful when episodes are short and full returns are observable.

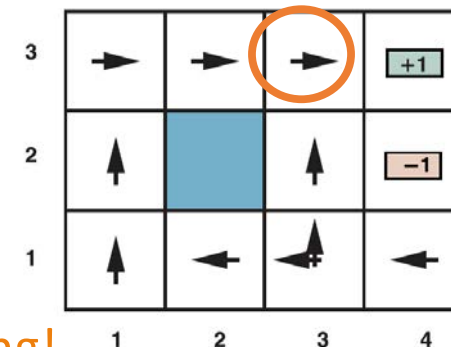
# Example: Temporal-Difference (TD) Learning

$(1, 1) \xrightarrow{-.04} (1, 2) \xrightarrow{-.04} (1, 3) \xrightarrow{-.04} (1, 2) \xrightarrow{-.04} (1, 3) \xrightarrow{-.04} (2, 3) \xrightarrow{.04} (3, 3) \xrightarrow{-.04} (4, 3) \xrightarrow{+1}$   
 $(1, 1) \xrightarrow{-.04} (1, 2) \xrightarrow{-.04} (1, 3) \xrightarrow{-.04} (2, 3) \xrightarrow{-.04} (3, 3) \xrightarrow{-.04} (3, 2) \xrightarrow{-.04} (3, 3) \xrightarrow{-.04} (4, 3) \xrightarrow{+1}$   
 $(1, 1) \xrightarrow{-.04} (2, 1) \xrightarrow{-.04} (3, 1) \xrightarrow{-.04} (3, 2) \xrightarrow{-.04} (4, 2) \xrightarrow{-1}$

After first trial:

- $U^\pi(1,3) = \frac{0.8+0.88}{2} = 0.84$
- $U^\pi(2,3) = 0.92$
- In the long run, must obey:
  - $U^\pi(1,3) = -0.04 + U^\pi(2,3)$
- Hence:
  - $U^\pi(1,3) = 0.88$  Current value of 0.84 needs updating!

Solving the MDP without learning a transition function.



# Example: Temporal Difference (TD) Learning

- Use Temporal Difference (TD) Update Rule

$$U^\pi(s) \leftarrow U^\pi(s) + \alpha[R + \gamma U^\pi(s') - U^\pi(s)]$$

From now on:  
Assume  $\gamma = 1$

$$U^\pi(1, 3) \leftarrow 0.84 + \alpha \cdot [-0.04 + 0.92 - 0.84] = 0.84 + \alpha \cdot 0.04$$

- If  $\alpha = 0.5$ , then:

$$U^\pi(1, 3) = 0.84 + 0.5 \cdot 0.04 = 0.86 \quad \text{Closer to the long run value 0.88!}$$

- What is  $\alpha$ ?

- $\alpha$  is the learning rate
- Controls how much each new experience affects the current utility estimate:
  - $\alpha = 1$ : full update using the new estimate
  - $\alpha = 0$ : ignore the new estimate
- Typically:  $0 < \alpha < 1$
- For convergence: use decreasing  $\alpha$ , e.g.,  $\alpha(n) = \frac{1}{n}$

# Temporal Difference Learning (TD)

- TD Update Rule (for a transition  $s \rightarrow s'$ ):

Update rule uses difference in utilities between successive states

$$U^\pi(s) \leftarrow U^\pi(s) + \alpha \left( \underbrace{R(s, \pi(s), s')}_{\text{TD Target}} + \underbrace{\gamma U^\pi(s') - U^\pi(s)}_{\text{TD Term or TD Error}} \right)$$

TD Target TD Term or TD Error

- TD Target:  $R + \gamma U^\pi(s')$
- TD Error:  $R(s) + \gamma U^\pi(s') - U^\pi(s)$
- Learning rate:  $\alpha$ , typically  $\alpha(n) = O(1/n)$  over visits
- Key Insights
  - Bootstrapping: Updates based on observed transitions and current estimate of  $U^\pi(s')$
  - Faster learning: TD updates after every step (vs. MC: end of episode)
  - No model required: Unlike Adaptive Dynamic Programming (ADP)
  - Converges: When every state is visited infinitely often, and  $\alpha$  decays appropriately

# Notes: TD-Learner (Model-Free RL)

**function** PASSIVE-TD-LEARNER(*percept*) **returns** an action

**inputs:** *percept*, a percept indicating the current state  $s'$  and reward signal  $r$

**persistent:**  $\pi$ , a fixed policy

$s$ , the previous state, initially null

$U$ , a table of utilities for states, initially empty

$N_s$ , a table of frequencies for states, initially zero

**if**  $s'$  is new **then**  $U[s'] \leftarrow 0$

**if**  $s$  is not null **then**

increment  $N_s[s]$

$U[s] \leftarrow U[s] + \alpha$

$s \leftarrow s'$

**return**  $\pi[s']$

TD Target   TD Term or TD Error

- TD needs only observed transitions to do the updates; no transition model is needed!
- Learn from every experience – update each time we experience  $s$ ,  $\pi(s)$ , and  $s'$

Source: RN Figure 22.4

# Notes: $n$ -step TD

- $n$ -step return (under policy  $\pi$ ):

$$G_t^{(n)} = R_t + \gamma R_{t+1} + \dots + \gamma^{n-1} R_{t+n-1} + \gamma^n U^\pi(S_{t+n})$$

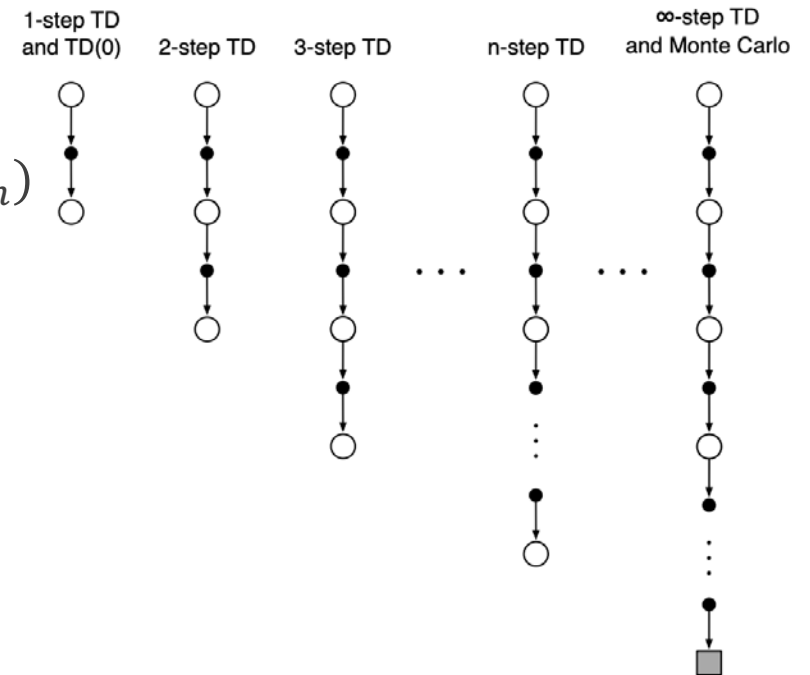
- $n$ -step TD update:

$$U^\pi(S_t) \leftarrow U^\pi(S_t) + \alpha(G_t^{(n)} - U^\pi(S_t))$$

where  $U^\pi(S_{t+n})$  is the estimated value at state  $S_{t+n}$

- Note:

- $n$ -step TD sets  $G_t^{(n)}$  as target for update
- TD sets 1-step return as target for update
- $\infty$ -step TD is Monte Carlo learning



Source: SB Figure 7.1

# Notes: TD( $\lambda$ )

- Average over different values of  $n$  instead of  $n$ -step returns

- Target for the update ( $\lambda$ -update):

$$u_t^\lambda = (1 - \lambda) \sum_{n=1}^{\infty} \lambda^{n-1} u_{t:t+n}$$

$$G_t^\lambda = (1 - \lambda) \sum_{n=1}^{\infty} \lambda^{n-1} G_t^{(n)}$$

- A weighted sum of  $n$ -step returns;  $n$ th step is weighted by  $\lambda^{n-1}$
- Can be computed efficiently by **eligibility traces** method

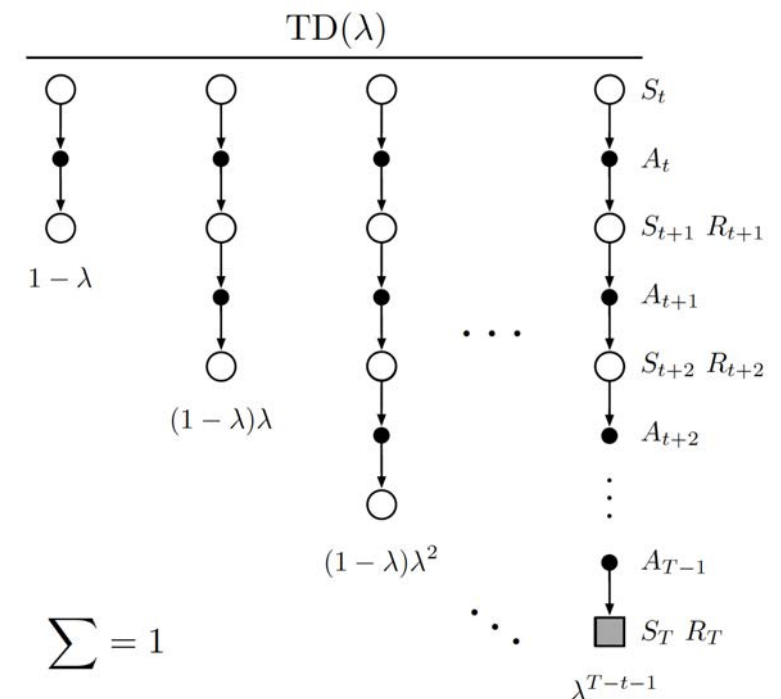
- Normalizing constant:

$$\sum_{n=1}^{\infty} \lambda^{n-1} = \sum_{n=0}^{\infty} \lambda^n = \frac{1}{1 - \lambda}$$

- Converges to TD as  $\lambda \rightarrow 0$ , and to MC as  $\lambda \rightarrow 1$

- How to prove this?

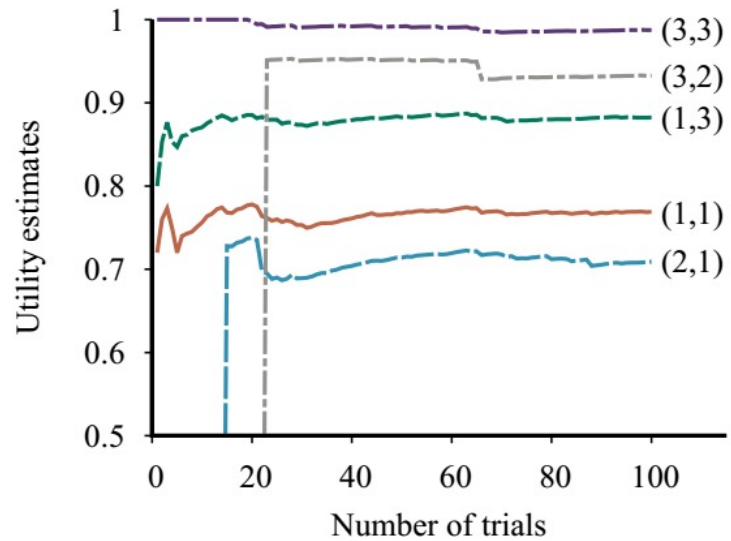
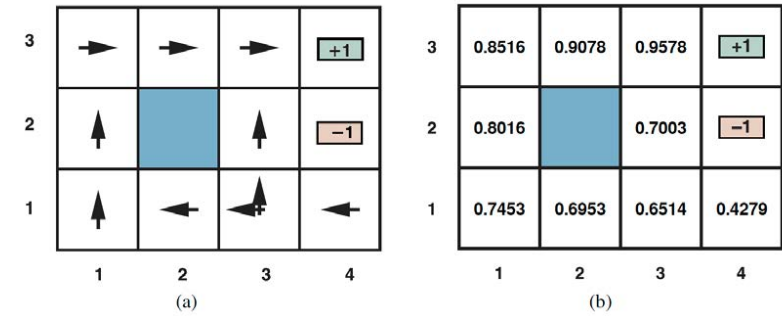
Sequential Decision Making



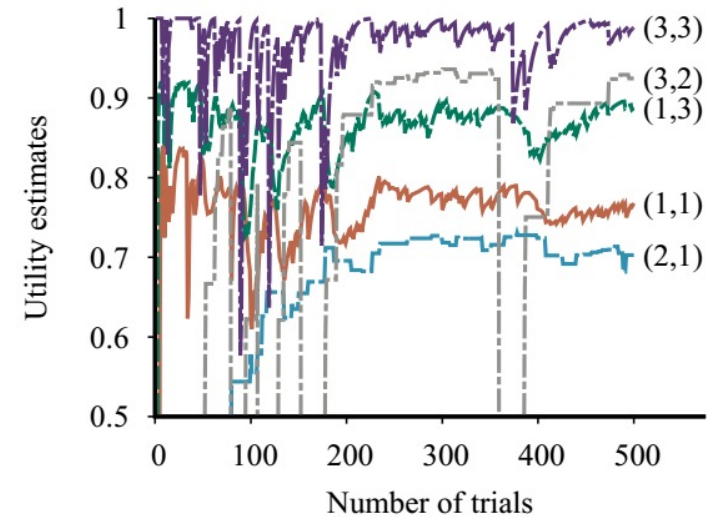
Source: SB Figure 12.1



# Example: ADP vs TD



Adaptive Dynamic Programming



Temporal Difference Learning

Source: RN Figure 22.3 (a) and 22.5 (a)

# ADP vs TD vs MC

| Aspect                          | ADP                                                                                              | TD(0)                                                                                    | MC                                                                                                                  |
|---------------------------------|--------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|
| Core equation                   | $U^\pi(s) = \sum_{s'} P(s' s, \pi(s)) [R(s, \pi(s), s') + \gamma U^\pi(s')] ]$                   | $U^\pi(s) \leftarrow U^\pi(s) + \alpha (R(s, \pi(s), s') + \gamma U^\pi(s') - U^\pi(s))$ | $U^\pi(s) = \frac{1}{N(s)} \sum_{i=1}^{N(s)} G_t^{(i)}$ where $G_t^{(i)} = \sum_{k=0}^{\infty} \gamma^k R(S_{t+k})$ |
| Update style                    | Makes each state agree with <b>all possible successors</b> using model $T$ (full Bellman backup) | Makes each state agree with its <b>observed successor</b> (bootstraps from estimates)    | Makes each state agree with the <b>full observed return</b> (no bootstrapping)                                      |
| Updates per observed transition | <b>Many/global</b> (sweeps all states using model)                                               | <b>One/local</b> (only visited state)                                                    | <b>One per state per episode</b> (after episode ends)                                                               |
| Model required                  | Needs <b>known/learned model</b> ( $T, R$ )                                                      | <b>X Model-free</b>                                                                      | <b>X Model-free</b>                                                                                                 |
| Data efficiency                 | <b>More data-efficient</b> (can simulate from model)                                             | <b>Moderate</b> (needs many samples but updates early)                                   | <b>Less data-efficient</b> (waits for full episodes)                                                                |
| Computation per update          | <b>Heavy</b> (solving Bellman equations or expectations)                                         | <b>Light</b> (incremental updates)                                                       | <b>Light</b> (averaging returns, but delayed)                                                                       |
| When updates occur              | After model learning or planning sweep                                                           | After <b>each step</b>                                                                   | After <b>each episode</b>                                                                                           |
| Bootstrapping?                  | Yes — uses current estimates of $U^\pi(s')$ from the model                                       | Yes — uses current estimate of $U^\pi(s')$ from samples                                  | No — uses actual returns $G$ , not current estimates                                                                |
| Summary                         | <b>Low variance, high bias</b> ; bootstraps from model                                           | <b>Lower variance, some bias</b> ; bootstraps from samples                               | <b>High variance, no bias</b> ; no bootstrapping                                                                    |



# Active Reinforcement Learning Learn $\pi^*$

Choose to act optimally based on prediction of utility or value of states

# Notes: Active Reinforcement Learning

- Main ideas

- $T$  (transition model) and  $R$  (reward function) are unknown
- Agent can choose any action  $a$  at each step
- Learns an optimal policy  $\pi^*$  that satisfies the Bellman optimality equations:

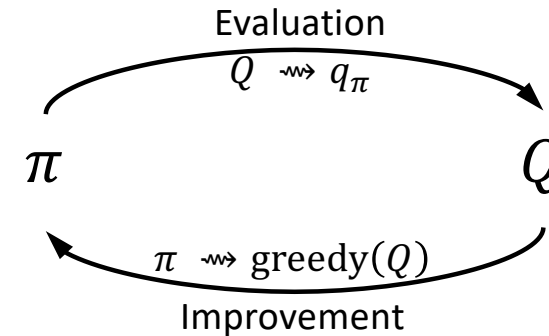
$$U(s) = \max_{a \in A(s)} \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma U(s')]$$

$$Q^*(s, a) = \sum_{s'} P(s'|s, a) \left[ R(s, a, s') + \gamma \max_{a'} Q^*(s', a') \right]$$

- Must balance exploration vs exploitation.
- If the learned model  $\hat{T}$  is inaccurate, the resulting  $\pi^*$  may be suboptimal (policy loss).

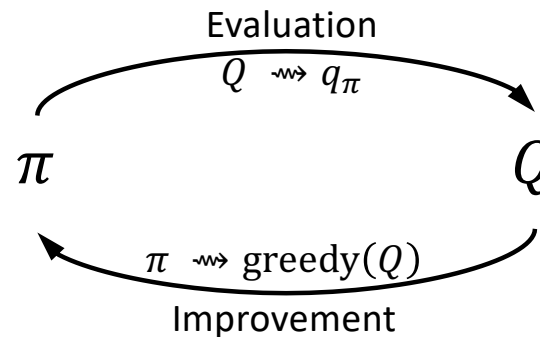
# Generalized Policy Iteration

- Generalized policy iteration (GPI):



- Repeatedly alternate between:
  - Policy evaluation — estimate utilities or values of the current policy (e.g., using **ADP**, Monte Carlo or Temporal Difference methods)
  - Policy improvement — act greedily with respect to the current utility estimates to refine the policy and gather better samples
- Key insight
  - These two processes **interleave and influence** each other
  - This feedback loop converges to the optimal policy

# Notes: GPI and Active RL Techniques



- How it connects

- Active ADP: does explicit evaluation + greedy improvement
- SARSA / Q-learning: do incremental TD evaluation; policy becomes greedier as estimates improve
- GPI helps us see these methods as different ways to realize the same loop



# Learning Optimal Policy

- Unknown transition dynamics, unknown reward functions
- **Model-based**
  - Learn the transition and reward functions, then solve the MDP ( $\pi^*$ )
  - Examples: **Active** Adaptive Dynamic Programming
- **Model-free**
  - Directly learn the optimal policy  $\pi^*$
  - Examples: Monte Carlo Control, Q-Learning, SARSA



# Active Adaptive Dynamic Programming

- Learn model  $T$  and  $R$  from passive ADP:

- Estimate transitions  $P(s'|s, a)$
- Estimate rewards  $R(s, a, s')$

- Learn optimal utilities via Bellman optimality equation:

$$U(s) = U^{\pi^*}(s) = \max_{a \in A(s)} \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma U(s')] = V(s)$$

- Extract optimal policy:

- $\pi^*(s) = \operatorname{argmax}_{a \in A(s)} \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma U(s')]$

# Notes: From Passive to Active ADP-Learner

**function** ~~PASSIVE~~-ADP-LEARNER(*percept*) **returns** an action

**inputs:** *percept*, a percept indicating the current state  $s'$  and reward signal  $r$

**persistent:**  ~~$\pi$ , a fixed policy~~  $\leftarrow \pi$  not fixed

Using tabular representation {  
 $mdp$ , an MDP with model  $P$ , rewards  $R$ , actions  $A$ , discount  $\gamma$   
 $U$ , a table of utilities for states, initially empty  
 $N_{s'|s,a}$ , a table of outcome count vectors indexed by state and action, initially zero  
 $s, a$ , the previous state and action, initially null

Count action  
oucomes

Reward  
function  $R$

Transition  
function  $T$

**if**  $s'$  is new **then**  $U[s'] \leftarrow 0$

**if**  $s$  is not null **then**

increment  $N_{s'|s,a}[s, a][s']$

$R[s, a, s'] \leftarrow r$

add  $a$  to  $A[s]$

$\mathbf{P}(\cdot \mid s, a) \leftarrow \text{NORMALIZE}(N_{s'|s,a}[s, a])$

~~$U \leftarrow \text{POLICY-EVALUATION}(\pi, U, mdp)$~~   $\leftarrow U \leftarrow \text{PolicyIteration}(\pi, U, mdp)$

$s, a \leftarrow s', \pi[s']$

**return**  $a$

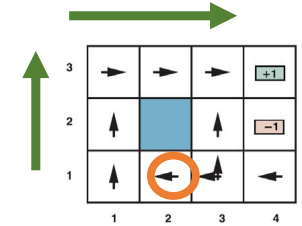
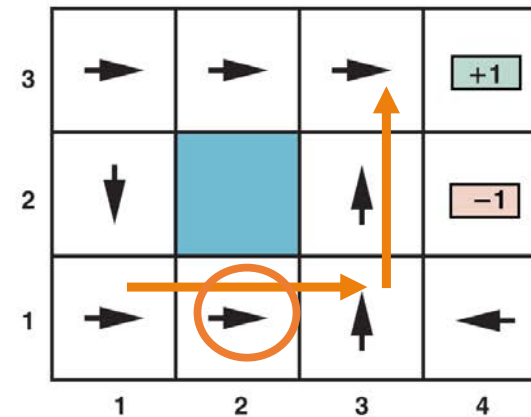
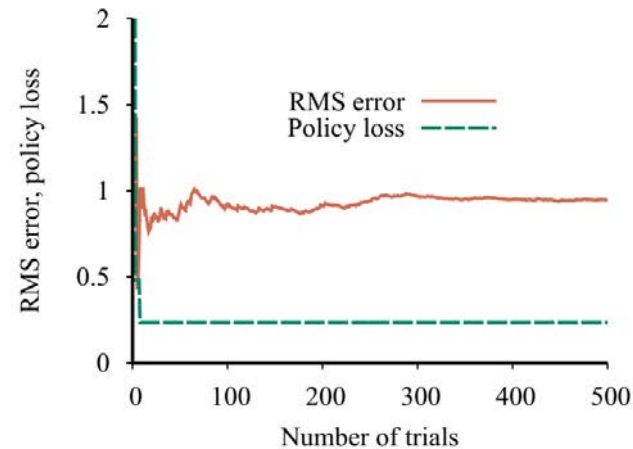
- $\pi$  is no longer fixed; changes as transitions & rewards are learned
- Replace POLICY-EVALUATION step by  $\pi \leftarrow \text{POLICY-ITERATION}(mdp)$  above

Source: RN Figure 22.2

# Greedy ADP Learner

RMS error in the utility estimates averaged over non-terminal states

Policy loss:  $U^{\pi^*} - U^{\pi}$



Source: RN Figure 22.6

- Learns model  $T$  and  $R$  through experience (passive ADP)
- Acts greedily: selects actions optimal under the current learned model
- May converge to a suboptimal policy
- Greedy behavior can limit exploration and future learning
- Actions influence both:
  - Immediate reward, and
  - Future data for improving the model

Sequential Decision Making

# Exploration Vs Exploitation

- Goal: Find an optimal policy that maximizes long-term rewards
- Exploration:
  - Try new actions to learn the true environment  $T \rightarrow$  may lead to better rewards later
- Exploitation:
  - Use the current model  $T$  to choose actions with the highest estimated rewards now
- Key challenges:
  - How to balance exploring enough to learn vs exploiting to earn?
  - When to stop exploring as knowledge improves?
- As the model becomes accurate, exploration should decrease

Hope to learn something new about the problem!



# Greedy in the Limit of Infinite Exploration

- GLIE: Balancing exploration and exploitation
  - Tries every action in every state infinitely often
  - Prevents missing the optimal action
  - Becomes greedy over time  $\rightarrow$  converges to optimal policy
- GLIE-based Exploration in Reinforcement Learning
  - $\epsilon$ -greedy exploration
    - Greedy action with  $1 - \epsilon$ ; random with  $\epsilon$
    - Converges slowly
- Improvements
  - Decay  $\epsilon$  (e.g.  $\epsilon = 1/t$ )
  - Use *exploration functions* (favor rarely tried actions)

# Optimism

- Encourages exploration by being optimistic about untried actions
- Use greedy selection w.r.t. an **optimistic utility estimate**  $U^+(s)$
- Update rule using an **exploration function**  $f(u, n)$ :

$$U^+(s) \leftarrow \max_a f \left( \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma U^+(s')], N(s, a) \right)$$

- $N(s, a)$ : number of times action  $a$  has been tried in state  $s$
- Less-tried actions get inflated estimates  $\rightarrow$  more likely to be explored



# Notes: Exploration Functions

- When to explore?
  - Goal: Decide when to explore
  - Random actions explore a fixed amount  $\rightarrow$  inefficient
  - Better: explore actions not yet proven bad
- Exploration function  $f(u, n)$ :
  - Takes value estimate  $u$  and visit count  $n$
  - Returns *optimistic utility* to guide greedy action choice
  - Balances greed (high  $u$ ) vs curiosity (low  $n$ )
  - Should *increase* with  $u$  and *decrease* with  $n$

## Example: An Exploration Function

- An  $f(u, n)$  increasing with  $u$  and decreasing with  $n$ :

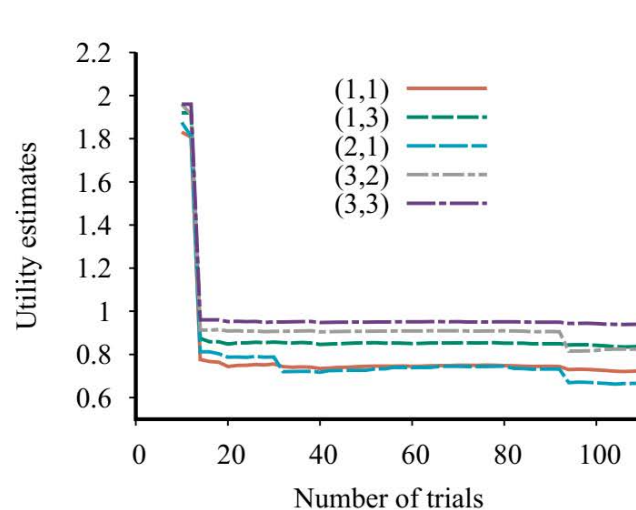
$$f(u, n) = \begin{cases} R^+ & \text{if } n < N_e \\ u & \text{otherwise} \end{cases}$$

Agent tries each state-action pairs  $N_e$  times

Where:

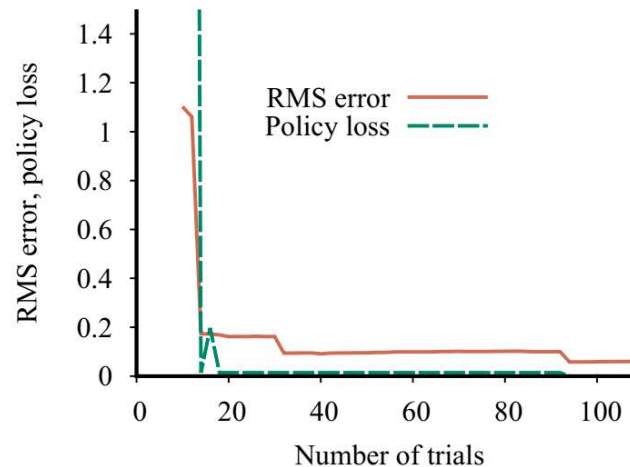
- Inputs: utility or value estimate  $u$  and visit count  $n$
- $R^+$ : an optimistic estimate of the best possible reward
- $N_e$ : a fixed parameter

# Example: Navigation in Grid World

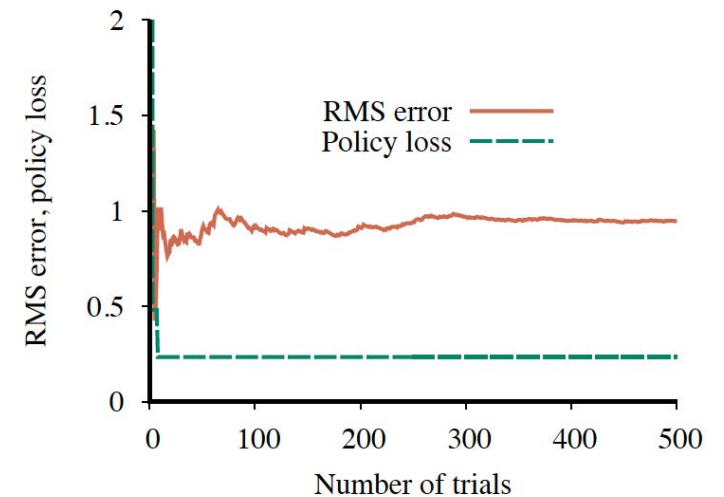


(a)

Exploring ADP



(b)



(a)

Greedy ADP

- Exploring ADP using exploration function with:  $R^+ = 2, N_e = 5$

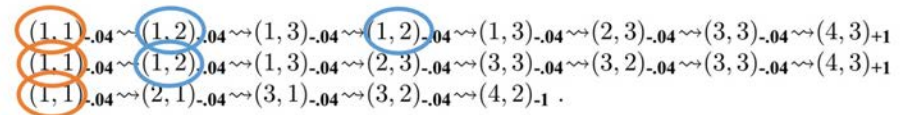
Source: RN Figure 22.6 (a) 22.7 (a) and (b)



# Learning Optimal Policy

- Unknown transition dynamics, unknown reward functions
- Model-based
  - Learn the transition and reward functions, then solve the MDP ( $\pi^*$ )
  - Examples: Active Adaptive Dynamic Programming
- **Model-free**
  - Directly learn the optimal policy  $\pi^*$
  - Examples: Monte Carlo Control, Q-Learning, SARSA

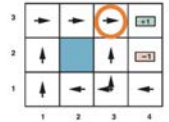
# Monte Carlo Control



- Each trial provides a “sample” of expected return for each state visited

$$U^\pi(1,1) = \frac{0.72+0.72-1.24}{3} = 0.067$$

$$U^\pi(1,2) = \frac{0.76+0.84+0.76}{3} = 0.79$$



## Policy evaluation

- Estimate action values using **Monte Carlo learning** (no model needed):

$$Q(s, a) \leftarrow \frac{1}{N(s, a)} \sum_{i=1}^{N(s, a)} G_t^{(i)}$$

- $G_t^{(i)}$ : return (discounted cumulative future rewards) from the  $i$ -th visit to  $(s, a)$
- $N(s, a)$ : number of times action  $a$  has been taken in state  $s$  (visits to  $(s, a)$ )
- This is the **Monte Carlo estimate** of  $Q^\pi(s, a)$  (sf.  $U^\pi(s)$ ) using episode returns

## Policy improvement

- Update policy to favor higher-value or underexplored actions:

$$\pi(s) = \arg \max_a f(Q(s, a), N(s, a))$$

- $f$ : **exploration function** encouraging less-visited actions (e.g., optimistic values)



# Active TD Control

- Goal:
  - Improve the policy while learning action-utilities  $Q(s, a)$  from experience
- Core idea:
  - Use TD updates to estimate action-utilities (no full episodes needed)
  - Use these updated utilities to make the policy greedier over time
- Two main algorithms:
  - SARSA (on-policy): learns from the actions actually taken
  - Q-learning (off-policy): learns from the best possible actions

# Notes: Active TD/TD Control

- Use Temporal-Difference (TD) learning for control:

- Action-value update:

$$Q(s, a) \leftarrow Q(s, a) + \alpha[R(s, a, s') + \gamma Q(s', a') - Q(s, a)]$$

- Policy update:

$$\pi(s) = \operatorname{argmax}_a Q(s, a) \text{ or } \pi(s) = \operatorname{argmax}_a f(Q(s, a), N(s, a))$$

$a'$ : action selected by estimation (target) policy

- Algorithms:

*SARSA*

- Use the **current**  $\pi$  to estimate Q
- On-policy: estimation (target) policy is **the same** as the behavior policy

*Q-Learning*

- Use the **greedy**  $\pi$  to estimate Q
- Off-policy: estimation (target) policy is **different** from the behavior policy

# SARSA

- State–Action–Reward–State–Action

Model free, on-policy method:

- Uses Temporal Difference (TD) for policy evaluation
- Uses  $\epsilon$ -greedy for action selection

- TD update:

$$Q(s, a) \leftarrow Q(s, a) + \alpha [R(s, a, s') + \gamma Q(s', a') - Q(s, a)]$$

TD Term or Error

- $a'$  : the next action actually taken in state  $s'$  (from the current policy  $\pi$ )
- This rule is applied after observing the full quintuple  $(s, a, r, s', a')$

- Convergence:

- With decreasing  $\epsilon$  and decreasing learning rates  $\alpha$ , and if all state–action pairs are visited infinitely often, SARSA converges to the optimal policy.

# Q-Learning: Learning Action-Utility Functions

- Q-learning

- **Model-free**: learns  $Q(s, a)$  directly from experience; no transition model or lookahead needed
- **Off-policy**: Based on the Bellman optimality equation for  $Q$ :

$$Q(s, a) = \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma \max_{a'} Q(s', a')] ]$$

- Learns via Temporal Difference (TD) updates:

$$Q(s, a) \leftarrow Q(s, a) + \alpha [R(s, a, s') + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

TD Term or Error

- **Convergence:**

- With decreasing  $\epsilon$  and decreasing learning rates  $\alpha$ , and if all state-action pairs are visited infinitely often, Q-Learning converges to the optimal policy.

# Notes: SARSA vs Q-Learning

```
1: controller SARSA( $S, A, \gamma, \alpha$ )
2:   Inputs
3:      $S$  is a set of states
4:      $A$  is a set of actions
5:      $\gamma$  the discount
6:      $\alpha$  is the step size
7:   Local
8:     real array  $Q[S, A]$ 
9:     state  $s, s'$ 
10:    action  $a, a'$ 
11:    initialize  $Q[S, A]$  arbitrarily
12:    observe current state  $s$ 
13:    select an action  $a$  using a policy based on  $Q$ 
14:    repeat
15:      do ( $a$ )
16:      observe reward  $r$  and state  $s'$ 
17:      select an action  $a'$  using a policy based on  $Q$ 
18:       $Q[s, a] := Q[s, a] + \alpha * (r + \gamma * Q[s', a'] - Q[s, a])$ 
19:       $s := s'$ 
20:       $a := a'$ 
21:    until termination
```

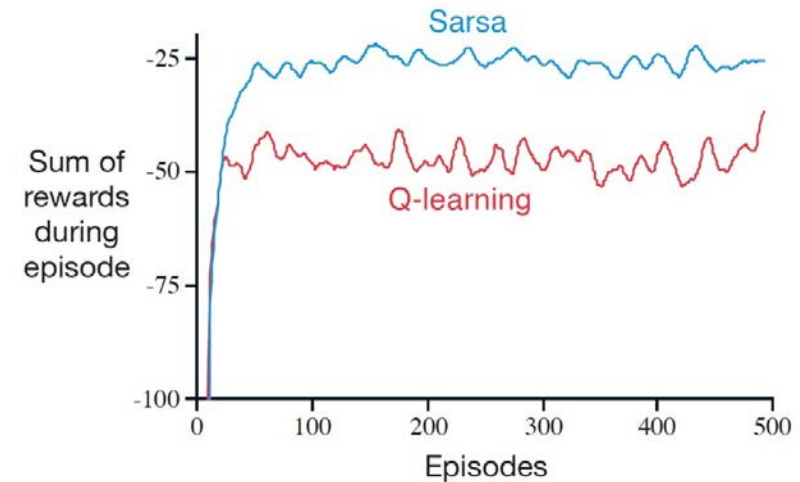
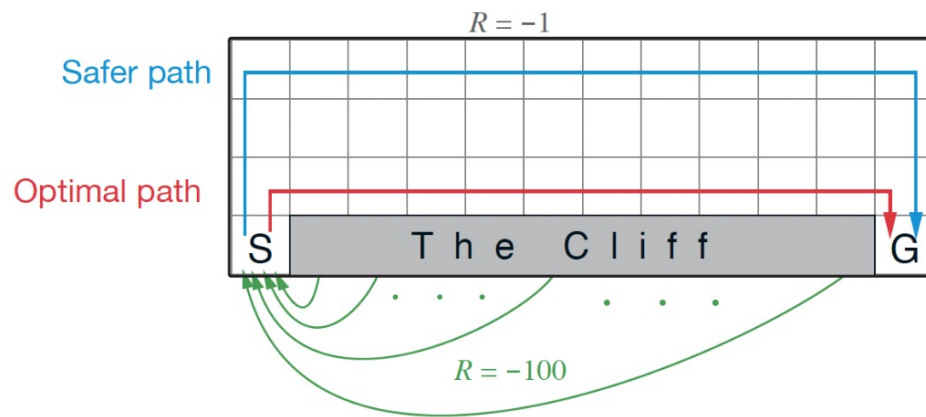
Figure 12.5: SARSA: on-policy reinforcement learning

```
1: controller Q-learning( $S, A, \gamma, \alpha$ )
2:   Inputs
3:      $S$  is a set of states
4:      $A$  is a set of actions
5:      $\gamma$  the discount
6:      $\alpha$  is the step size
7:   Local
8:     real array  $Q[S, A]$ 
9:     states  $s, s'$ 
10:    action  $a$ 
11:    initialize  $Q[S, A]$  arbitrarily
12:    observe current state  $s$ 
13:    repeat
14:      select an action  $a$ 
15:      do ( $a$ )
16:      observe reward  $r$  and state  $s'$ 
17:       $Q[s, a] := Q[s, a] + \alpha * (r + \gamma * \max_{a'} Q[s', a'] - Q[s, a])$ 
18:       $s := s'$ 
19:    until termination
```

Figure 12.3: Q-learning controller

Source: Poole and Macwark, AI Foundations of Computational Agents, 2e, 2018

# Example: Cliff Walking



- Actions: left, right, up, down
- Reward:  $-1$  for each step;  $-100$  if you fall off
- SARSA &  $Q$ -learning used with  $\epsilon$ -greedy;  $\epsilon = 0.1$ 
  - Why does SARSA outperform  $Q$ -learning?
  - What happens if GLIE exploration is used?

Source: SB: Example 6.6

# SARSA vs Q-Learning

|                           | SARSA                                                                                    | Q-learning                                                                                 |
|---------------------------|------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|
| <b>Type</b>               | On-policy — learns from actions actually taken by the current (exploring) policy         | Off-policy — learns as if always choosing the best action                                  |
| <b>Update</b>             | $Q(s, a) \leftarrow Q(s, a) + \alpha (R(s, a, s') + \gamma Q(s', a') - Q(s, a))$         | $Q(s, a) \leftarrow Q(s, a) + \alpha (R(s, a, s') + \gamma \max_{a'} Q(s', a') - Q(s, a))$ |
| <b>Target</b>             | Uses $Q(s', a')$ from the <b>next action actually taken</b>                              | Uses $\max_{a'} Q(s', a')$ from the <b>best possible action</b>                            |
| <b>Exploration effect</b> | Penalizes bad exploratory actions → safer, more cautious                                 | Ignores outcomes of exploratory actions → more optimistic                                  |
| <b>Learning behavior</b>  | Incorporates negative outcomes → can be <b>slower but more stable</b>                    | Ignores negative exploratory outcomes → <b>faster but riskier</b>                          |
| <b>Similarity</b>         | Behave the same when the agent <b>never explores</b> (always takes the best action)      | Behave the same when the agent <b>never explores</b> (always takes the best action)        |
| <b>Convergence</b>        | Converges if $\epsilon$ and $\alpha$ decay and all $(s, a)$ are visited infinitely often | Converges if $\epsilon$ and $\alpha$ decay and all $(s, a)$ are visited infinitely often   |



# Summary: Reinforcement Learning

- **Passive Learning — fixed policy (policy evaluation)**
  - [MB] Adaptive dynamic programming (ADP)
  - [MF] Monte Carlo prediction (MC): direct value (utility) estimation
  - [MF] Temporal-difference (TD) prediction
- **Active Learning — optimal policy (control)**
  - [MB] Active ADP (model-based control)
  - [MF] Monte Carlo control
  - [MF] Active TD methods (TD control): Q-learning, SARSA



# Homework

- Readings:
  - RN: 22.1, 22.2. 22.3.1, 22.3.2
  - *SB*: 6.2, 6.4, 6.5 (Optional)
- Some classical papers and recent surveys on RL  
(Journal articles publicly available online or through NUS Library e-Resources)
  - L. P. Kaelbling, M. L. Littman, & A. W. Moore. [Reinforcement Learning: A Survey](#). JAIR, 4:237-285, 1996
  - Wirth, C., et al., [A survey of preference-based reinforcement learning methods](#). J. Mach. Learn. Res., 2017. **18**(1): p. 4945–4990.